Lezione 7 · Token bucket e leaky bucket

Network Modeling · 20 ottobre 2026 · Appunti integrati

Questi appunti seguono la lezione: richiamo del token bucket, policer a due rate e tre colori, leaky bucket, esercizio a dente di sega e dimensionamento di uno shaper con vincoli di burst, tolleranza e ritardo. Le spiegazioni del prof e gli appunti manoscritti sono la base; il capitolo serve a completare gli stessi argomenti.

1. Prima distinzione: controllare o modificare il traffico

Un **policer**, nel modello presentato dal prof, verifica se un pacchetto rispetta il contratto e lo **marca**. Saranno altri moduli a decidere se inoltrarlo con meno priorità, trattarlo come best effort oppure scartarlo.

Uno **shaper** modifica invece il traffico in uscita: può trattenere i pacchetti in una coda e farli uscire più lentamente. Se la coda finita si riempie, il traffico in eccesso viene perso.

> **Idea di base.** I token sono crediti per trasmettere; i buffer contengono dati in attesa. Non sono la stessa cosa.

Notazione e unità

Simbolo	Significato
ρ	Generazione di token al secondo
b e $z(t)$	Capacità del bucket di token e token disponibili
r	Rate massimo del **leaky bucket**
B e $q(t)$	Capacità di un buffer dati e dati presenti
$G(t), G_{\text{out}}(t)$	Rate di ingresso e uscita

Negli appunti il prof usa ρ anche per il leaky bucket: qui lo chiamiamo r per non confonderlo con la generazione di token. Assumiamo **1 token** = **permesso di trasmettere 1 byte**. In questo caso ρ corrisponde numericamente a un rate in byte/s. Quando un esercizio usa bit/s, convertiamo anche la memoria in bit.

Token bucket a un rate e due colori

Il bucket riceve token al rate ρ e ne contiene al massimo b. Per un pacchetto di W byte:

- Se $z(t) \geq W$, si consumano W token e il pacchetto è **conformant**.
- Altrimenti il pacchetto è **exceed** e **non si consumano token**.

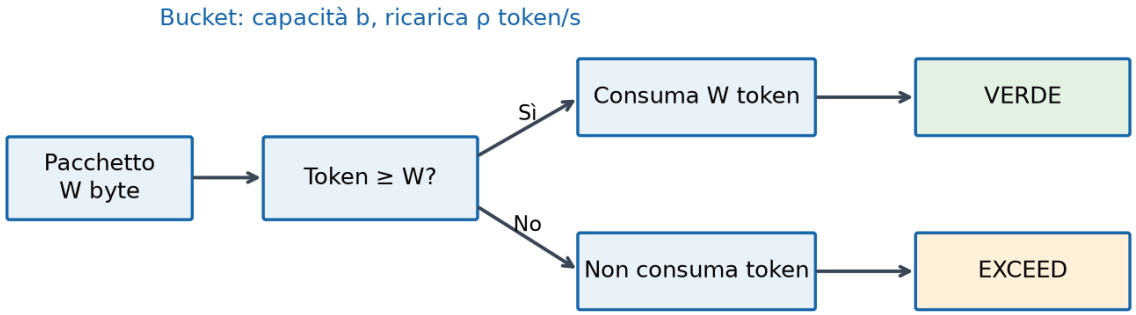


Figura 1 - Decisione del policer a un rate.

Come leggere lo schema. La domanda centrale riguarda i crediti, non lo spazio in un buffer. Il ramo negativo assegna una marca; non impone da solo uno scarto. Nel contratto, **CIR** (*Committed Information Rate*) corrisponde al rate concordato e **CBS** (*Committed Burst Size*) alla capacità di crediti, ossia b nel modello minimo.

2. Crediti, burst e policer a due rate

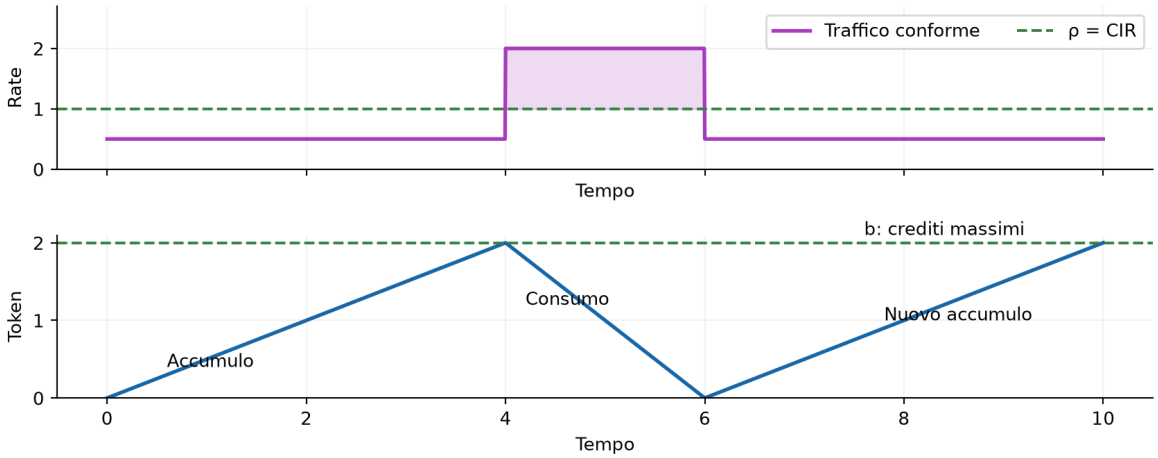


Figura 2 - Traffico conforme e crediti disponibili, in unità illustrative.

Significato del grafico. Quando il rate è sotto ρ, i token si accumulano. Durante il burst, il rate supera ρ e il bucket si svuota: si spendono crediti risparmiati prima. Una volta raggiunto b, i nuovi token inutilizzati sono persi; aspettare ancora non aumenta la riserva.

Il token bucket semplice controlla il rate conforme di lungo periodo e l'eccesso accumulabile, ma **non pone da solo un limite al picco istantaneo**. I crediti possono essere spesi molto rapidamente. Inoltre, b limita l'eccesso rispetto ai token prodotti nel frattempo, non tutti i byte trasmessi durante un burst di durata non nulla.

Two-rate three-color: due controlli, tre risultati

Il prof introduce un secondo bucket per distinguere un superamento del rate concordato da una violazione anche del profilo di picco.

Bucket	Ricarica	Capacità	Cosa controlla
Committed	CIR	CBS	Profilo del traffico conforme
Peak	PIR, con PIR $\geq$ CIR	PBS	Profilo di picco, con propria tolleranza al burst

Si controlla **prima il bucket peak**, poi quello committed. Per un pacchetto di W byte:

Crediti disponibili	Colore	Token consumati
Peak $< W$	**Rosso: violating**	Nessuno
Peak $\geq W$, committed $< W$	**Giallo: exceed**	W solo dal peak
Entrambi $\geq W$	**Verde: conformant**	W da entrambi

Esempio qualitativo del prof, nel modello fluido. Partendo con crediti disponibili, se la sorgente mantiene $\lambda \leq$ CIR, il traffico può restare verde. Se CIR $< \lambda \leq$ PIR, i crediti committed diminuiscono e compare traffico giallo. Se $\lambda >$ PIR, possono esaurirsi anche i crediti peak e comparire pacchetti rossi.

Non significa che, dopo il primo esaurimento, ogni pacchetto abbia necessariamente lo stesso colore: i token continuano a essere generati. L'ordine esatto di esaurimento dipende anche da CBS, PBS e dai livelli iniziali. Il percorso verde-giallo-rosso descritto a voce è un esempio intuitivo, non una regola indipendente dai parametri.

Anche questo è un **policer**: i colori classificano i pacchetti. Per imporre un limite rigido al rate in uscita serve uno shaper; il bucket peak conserva comunque una tolleranza finita al burst.

3. Leaky bucket: una coda con uscita limitata

Immaginiamo un recipiente con un foro: arrivano dati, si accumulano se necessario ed escono al massimo al rate r. Il recipiente è un **buffer dati di capacità** B, non un portafoglio di token.

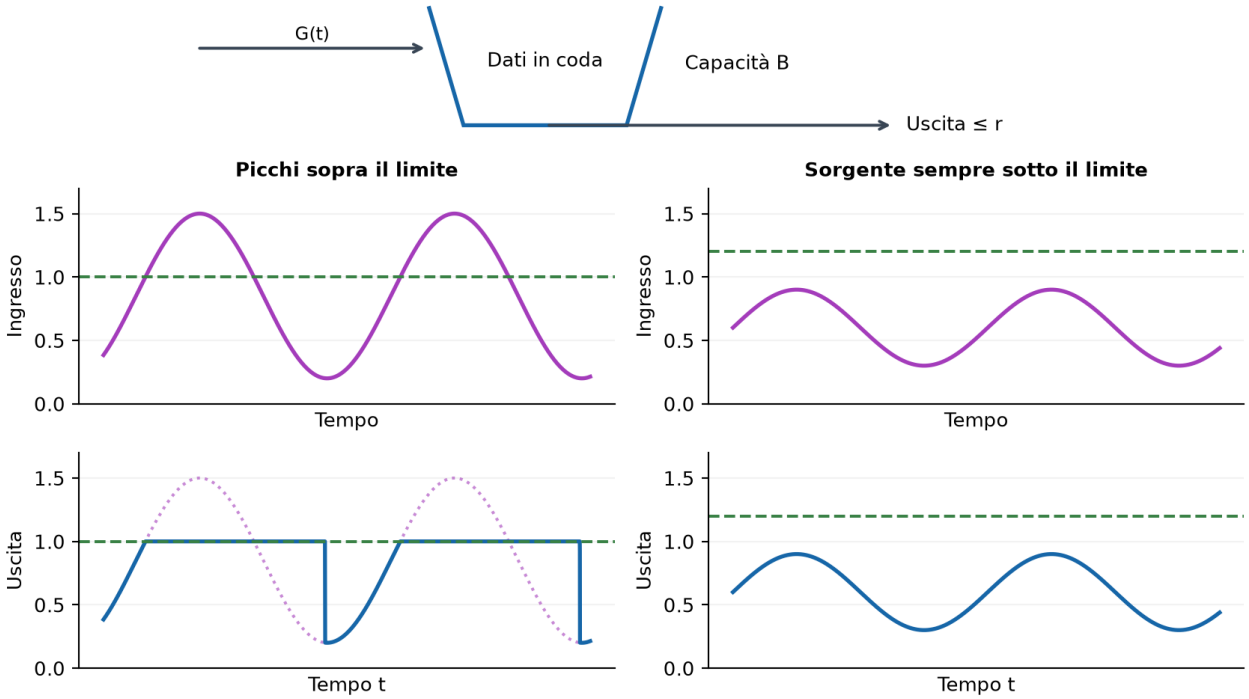


Figura 3 - Recipiente e quattro grafici di ingresso/uscita del leaky bucket.

Pannelli di destra. Con buffer inizialmente vuoto e ingresso sempre sotto r, non si forma coda: l'uscita coincide con l'ingresso.

Pannelli di sinistra. Quando l'ingresso supera r, l'uscita si ferma a r e l'eccesso viene accodato. Quando l'ingresso torna sotto r, **l'uscita resta a** r **finché ci sono dati arretrati**. Solo dopo lo svuotamento torna a seguire la sorgente. La linea viola tratteggiata aiuta a confrontare ingresso e uscita: il picco viene distribuito su un tempo più lungo.

La regola, nel modello fluido, è:

$$G_{\text{out}}(t) = \begin{cases} \min\{G(t), r\}, & q(t) = 0, \\ r, & q(t) > 0. \end{cases}$$

Quando $0 < q(t) < B$, la coda varia secondo **ingressi meno uscite**:

$$\frac{dq(t)}{dt} = G(t) - r.$$

Se $q(t) = B$ e $G(t) > r$, il buffer non può crescere: si scarta l'eccesso. Un buffer più grande assorbe un picco più lungo, ma non risolve un sovraccarico permanente.

> **Da ricordare.** "Rate costante" significa che il leaky bucket serve al rate r **quando ha dati da trasmettere**. Non produce traffico fittizio quando la coda è vuota.

4. Esercizio: ingresso periodico a dente di sega

Il prof propone un leaky bucket con $r = 1$ Mbit/s e $B = 1\,\text{kB} = 8000$ bit, usando il kilobyte decimale. Il rate della sorgente cresce linearmente da zero a R e riparte ogni T secondi:

$$G(t) = \frac{R}{T}\,\mathrm{mod}(t, T) = R\left(\frac{t}{T} - \left\lfloor \frac{t}{T} \right\rfloor\right).$$

Qui $\mathrm{mod}(t, T) = t - T\lfloor t/T \rfloor \in [0, T)$ e il *floor* prende la parte intera inferiore. Il fattore $1/T$ è necessario: R **è un rate**, non una pendenza. Il prof corregge questa normalizzazione durante la lezione.

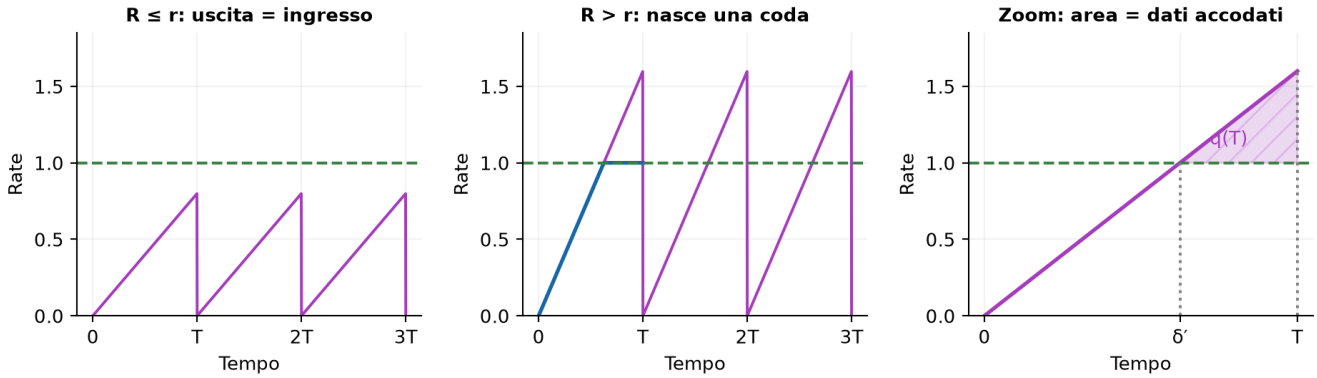


Figura 4 - Dente di sega sotto il limite, sopra il limite e dettaglio dell'accumulo.

Sinistra: $R \le r$**.** Partendo con buffer vuoto, $G_{\text{out}}(t) = G(t)$: non c'è coda. L'uscita non è costantemente R; R è soltanto il valore massimo raggiunto dalla rampa.

Centro: $R > r$**.** Nel primo periodo l'uscita segue la rampa fino all'istante in cui $G(t) = r$, poi rimane a r:

$$\delta' = \frac{rT}{R}, \qquad \delta = T - \delta' = \frac{R - r}{R}T.$$

Destra: zoom del triangolo. L'area tra ingresso e uscita misura i dati accodati, perché un rate moltiplicato per un tempo dà una quantità di dati. A fine periodo:

$$q(T) = \frac{(R - r)\delta}{2} = \boxed{\frac{(R - r)^2 T}{2R}}.$$

Questo risultato assume **coda inizialmente vuota e nessun overflow nel primo periodo**. Serve quindi $B \ge q(T)$; altrimenti la coda raggiunge B prima di T e parte dell'eccesso viene scartata.

L'intero triangolo di ingresso ha invece area $RT/2$: sono i dati generati in un periodo. Quindi il rate medio è $\overline{G} = R/2$. La scritta provvisoria "RT per periodo" negli appunti va corretta con il fattore $1/2$.

5. Dopo il salto a zero: quando si svuota la coda?

A $t = T$ l'ingresso riparte da zero, ma il buffer contiene ancora $q(T)$ dati. L'uscita resta quindi a r. Poniamo $t = T + \varepsilon$, con $0 \le \varepsilon < T$.

Finché la coda rimane positiva e non ci sono perdite:

$$q(T+\varepsilon) = q(T) + \underbrace{\int_0^{\varepsilon} \frac{R}{T} u\, du}_{\text{nuovi dati}} - \underbrace{r\varepsilon}_{\text{dati trasmessi}} = q(T) + \frac{R}{2T}\varepsilon^2 - r\varepsilon.$$

Correzione del passaggio scritto a lezione: non si può usare $G(\varepsilon)\varepsilon$, perché durante l'intervallo il rate cresce da zero. L'area è un triangolo, quindi compare ancora $1/2$.

Per trovare lo svuotamento imponiamo $q(T+\varepsilon) = 0$ e prendiamo la **prima radice fisicamente valida**:

$$\varepsilon_0 = \frac{T}{R}\left[r - \sqrt{r^2 - \frac{2R}{T}q(T)}\right] = \frac{T}{R}\left[r - \sqrt{R(2r-R)}\right].$$

Per $r < R < 2r$, questa radice soddisfa $0 < \varepsilon_0 < \delta'$. La seconda radice della parabola non descrive la coda: dopo lo svuotamento non possiamo continuare la formula fino a valori negativi.

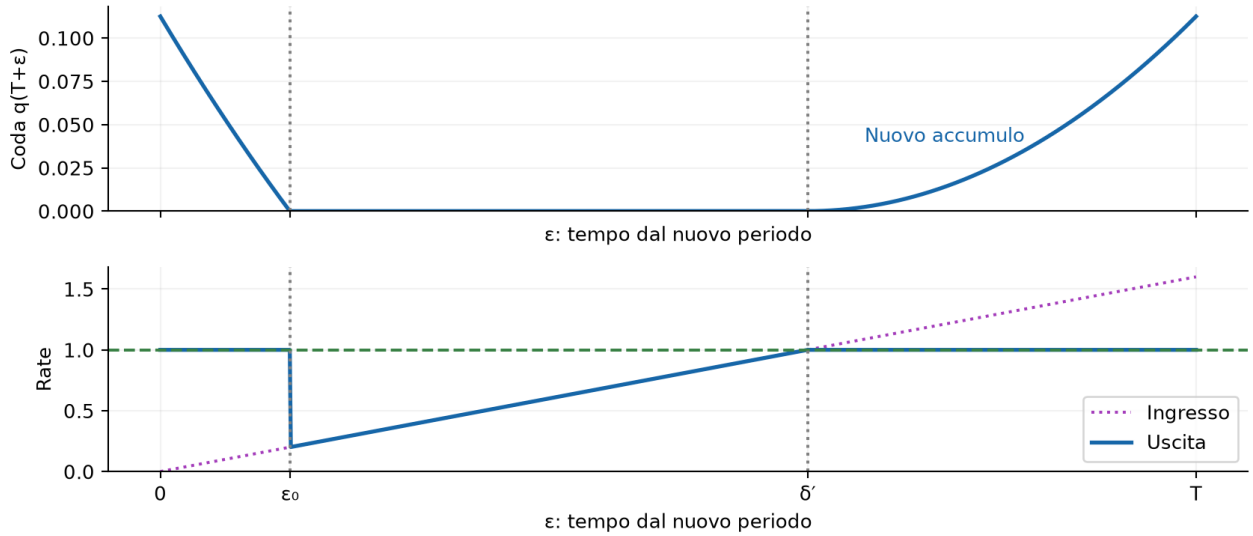


Figura 5 - Coda e uscita nel periodo successivo, per r < R < 2r.

Grafico superiore. La coda scende con andamento parabolico fino a ε_0, resta nulla fino a $\delta' = rT/R$, poi ricomincia a crescere quando la nuova rampa supera r. Il disegno manoscritto era schematico: il tratto a zero non continua fino a fine periodo.

Grafico inferiore. Fino a ε_0 l'uscita vale r. Quando finiscono i dati arretrati, scende al rate istantaneo della sorgente, segue la rampa e torna a r quando l'ingresso raggiunge il limite. Questo spiega il "gradino verso il basso" disegnato dal prof.

6. Stabilità e perdite: completamento dell'esercizio

Il confronto decisivo è tra il rate medio $R/2$ e il servizio r:

Condizione	Comportamento, con buffer inizialmente vuoto
$R \leq r$	Nessuna coda, uscita uguale all'ingresso
$r < R < 2r$	La coda si svuota in ogni ciclo; nessuna perdita se $B \geq q(T)$
$R = 2r$	Caso limite: la coda tocca zero a $\varepsilon_0 = T/2$ e ricresce subito
$R > 2r$	Un buffer illimitato accumula dati ciclo dopo ciclo; con buffer finito compaiono perdite

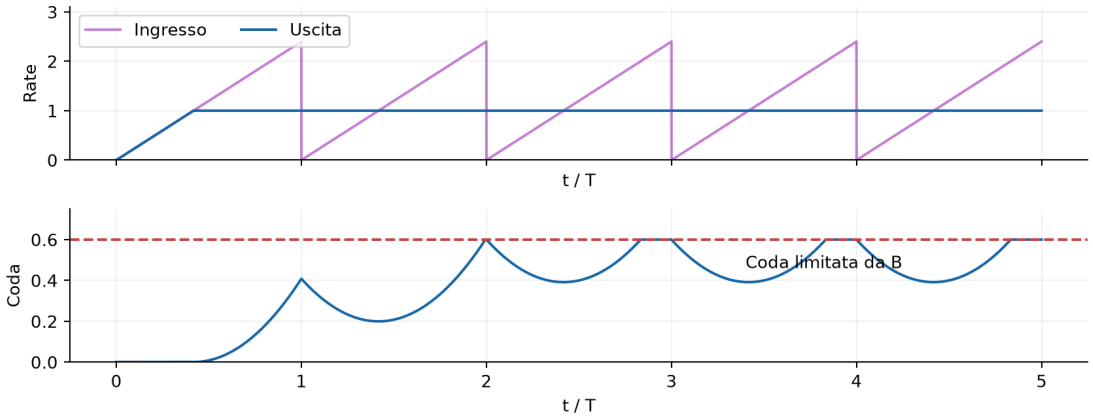


Figura 6 - Esempio di sovraccarico periodico con buffer finito.

Sopra: l'ingresso continua a oscillare; nell'esempio, dopo il transitorio, c'è sempre abbastanza backlog per mantenere l'uscita a r. **Sotto:** il buffer tocca il limite B e poi si svuota parzialmente a ogni ciclo. Dunque non va interpretata letteralmente l'annotazione "$q(t) \to B$": con ingresso periodico, la coda può oscillare anche quando ci sono perdite. Con un buffer molto piccolo può perfino svuotarsi e far scendere temporaneamente l'uscita sotto r.

Esempio più semplice del prof: sorgente CBR

Se l'ingresso è costante, $\lambda = 1{,}5r$, la coda inizialmente vuota cresce al rate $\lambda - r = 0{,}5r$. Diventa piena dopo:

$$t_{\text{pieno}} = \frac{B}{\lambda - r}.$$

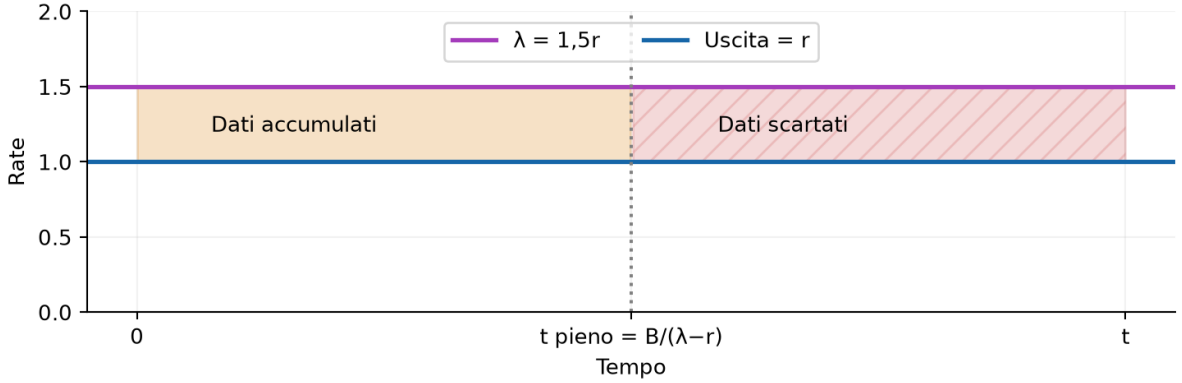


Figura 7 - Eccesso prima accumulato e poi scartato, con ingresso costante.

Significato delle aree. Prima di t_{pieno}, l'area tra λ e r riempie il buffer. Dopo, lo stesso eccesso viene perso. L'uscita vale sempre r; il rate scartato a regime è $0{,}5r$, cioè **un terzo dell'ingresso**. Diversamente dal dente di sega, qui il buffer rimane pieno perché l'ingresso non scende mai.

7. Token bucket come shaper: burst e grace period

La seconda parte della lezione combina un token bucket con buffer dati e link di uscita. La sorgente specifica un **SLA** (*Service Level Agreement*), ossia i requisiti del servizio:

Parametro	Richiesta della sorgente
λ_a	Rate medio di lungo periodo
$\lambda_p > \lambda_a$	Rate di picco
T_b	Durata del burst da far passare al picco attraverso il TBF
T_g	Ulteriore tempo al picco tollerato senza perdita: **grace period**
d_{SLA}	Massimo ritardo accettabile per dati non scartati

Il **burst** usa i crediti già disponibili e attraversa il TBF a λ_p. Nel **grace period** i crediti sono finiti: la sorgente può ancora generare a λ_p, ma il TBF inoltra a ρ. La differenza deve aspettare in una coda.

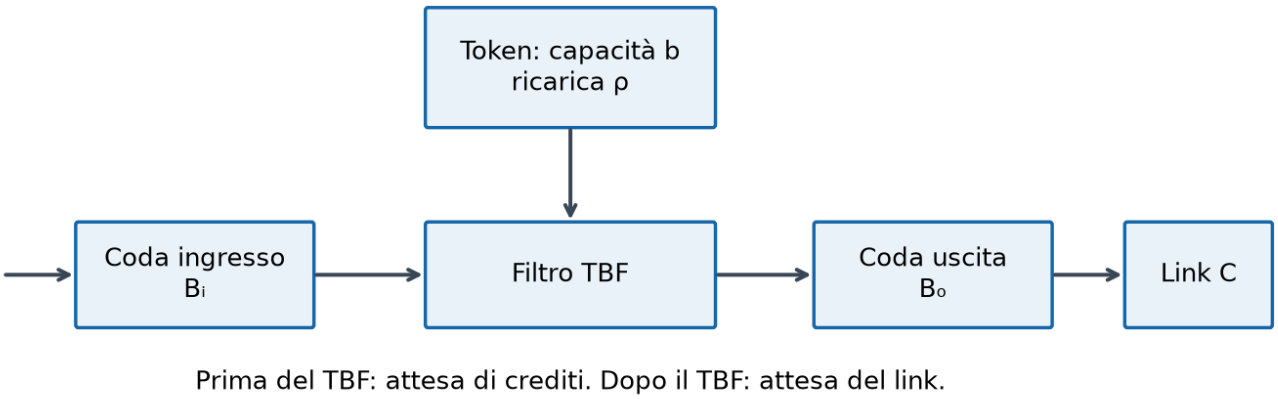


Figura 8 - Token bucket, buffer di ingresso, buffer di uscita e link.

Come leggere lo schema, da sinistra a destra.

- Il buffer B_i conserva dati che aspettano i token.
- Il bucket b conserva solo crediti; il filtro li consuma per autorizzare il passaggio dei dati.
- Il buffer B_o conserva dati già passati dal filtro, ma in attesa di trasmissione sul link.
- Il link trasmette al rate massimo C.

La coppia **buffer di uscita + link** si comporta come un leaky bucket di rate C. Ecco il senso della combinazione: il TBF consente burst limitati dai crediti; il link impone il limite fisico finale. Se $C < \lambda_p$, un burst attraversa il TBF al picco ma **non esce dal link al picco**: si accoda in B_o.

Ipotesi usate nei calcoli del prof

Usiamo un **modello fluido**: i dati sono un flusso continuo, senza confini tra pacchetti. All'inizio del burst il bucket è pieno e i due buffer sono vuoti. Le code sono FIFO, la sorgente non supera λ_p e l'inoltro del TBF nel modello analizzato è limitato a quel picco. Studiamo il caso peggiore in cui la sorgente resta al picco per tutto il burst e il grace period.

Il limite di picco è un'ipotesi/configurazione aggiuntiva del sistema analizzato: il bucket semplice, con i soli ρ e b, non lo impone da solo. I ritardi calcolati sono ritardi di accodamento del modello, senza propagazione e dettagli di serializzazione dei pacchetti.

8. Dimensionare token e buffer con un bilancio

La regola comune è: **quantità accumulata = differenza tra rate × durata**. Per un dimensionamento minimo scegliamo $\rho = \lambda_a$; in generale serve $\rho \geq \lambda_a$. Consideriamo $\rho < \lambda_p$ e un link con $C > \rho$, così può smaltire anche il traffico successivo al burst.

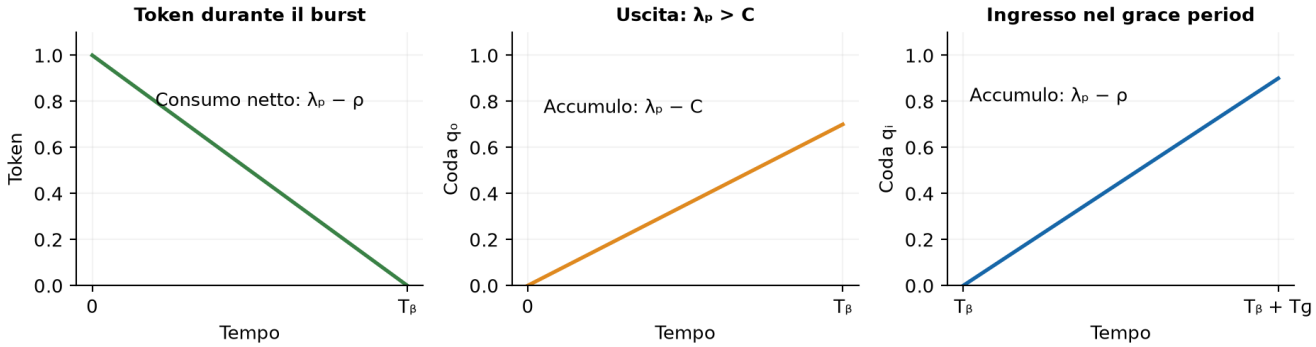


Figura 9 - I tre bilanci: crediti consumati, coda sul link, coda in ingresso.

Crediti necessari al burst

Pannello sinistro. I token escono al rate λ_p, ma continuano a entrare al rate ρ. Il livello scende quindi a $\lambda_p - \rho$, non a λ_p:

$$T_{\text{esaurimento}} = \frac{b}{\lambda_p - \rho}.$$

Per consentire almeno T_b, serve $b \geq (\lambda_p - \rho)T_b$. Se vogliamo che i crediti finiscano esattamente alla fine del burst concordato, scegliamo:

$$\boxed{b = (\lambda_p - \rho)T_b}.$$

Le formule successive usano questa scelta. Aumentando b, aumenta anche il burst effettivo da considerare nel dimensionamento dell'uscita.

Buffer di uscita

Pannello centrale. Se $\rho < C < \lambda_p$, durante T_b entrano $\lambda_p T_b$ byte ed escono CT_b byte. Quindi:

$$\boxed{B_o \geq (\lambda_p - C)T_b}.$$

Se invece $C \geq \lambda_p$, non si accumula coda all'uscita e nel modello fluido basta $B_o = 0$. Nella pratica rimane memoria per gestire i pacchetti. Comprare un link molto più veloce del picco evita questa coda, ma può essere uno spreco di capacità: è il compromesso evidenziato dal prof.

Buffer di ingresso e grace period

Pannello destro. Dopo l'esaurimento dei token arrivano λ_p byte/s, ma ne passano solo ρ. Per tollerare un grace period T_g:

$$\boxed{B_i \geq (\lambda_p - \rho)T_g}.$$

Se scegliamo l'uguaglianza, il buffer si riempie a $t_p = T_b + T_g$. Se la sorgente insiste al picco oltre quell'istante, l'eccesso viene scartato. Riempire un buffer dati è diverso dall'esaurire i token: il primo porta a perdite, il secondo riduce il rate di inoltro.

9. Ritardo massimo con link veloce: $C \geq \lambda_p$

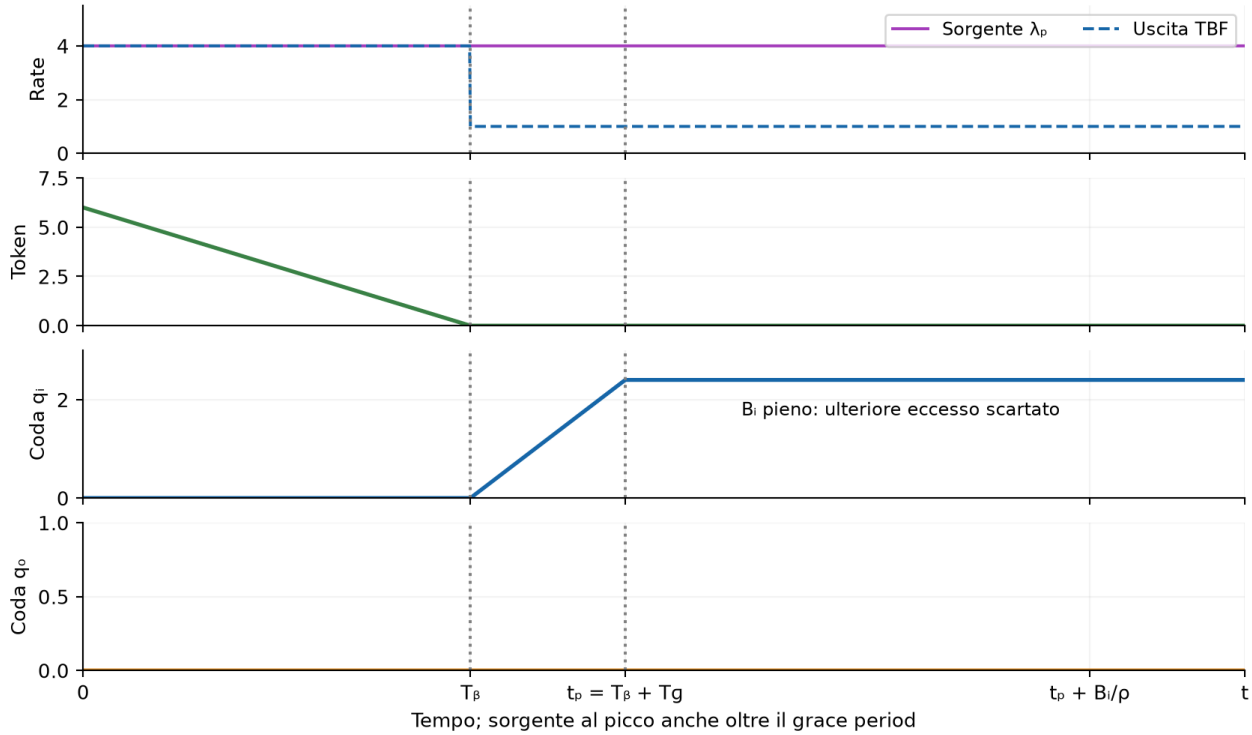


Figura 10 - Rate, token, coda di ingresso e coda di uscita con link veloce.

Questi sono i grafici descritti dal prof nella parte lasciata vuota nell'ultima pagina degli appunti. Qui la sorgente continua al picco anche oltre il grace period, per mostrare saturazione e scarti.

Primo grafico: rate. Fino a T_b il TBF inoltra al picco. Dopo T_b, esauriti i token, inoltra solo a ρ.

Secondo grafico: token. La riserva scende linearmente da b a zero al rate $\lambda_p - \rho$. Dopo, i nuovi token vengono consumati appena prodotti: il livello resta zero.

Terzo grafico: ingresso. La coda è vuota durante il burst; cresce nel grace period al rate $\lambda_p - \rho$ e raggiunge B_i a t_p. Se il picco continua, rimane piena e l'ulteriore eccesso viene scartato.

Quarto grafico: uscita. La coda è sempre nulla perché il link riesce a trasmettere almeno quanto riceve dal TBF.

Il dato nel caso peggiore è quello accettato in fondo a una coda d'ingresso piena. Davanti ha circa B_i byte, serviti al rate ρ:

$$\boxed{d_{\max} = \frac{B_i}{\rho}}.$$

Per rispettare il ritardo richiesto e garantire il grace period devono valere insieme:

$$(\lambda_p - \rho)T_g \leq B_i \leq \rho d_{\text{SLA}}.$$

Quindi più tolleranza richiede più memoria, ma può anche aumentare l'attesa massima. Se la sorgente riduce il rate sotto ρ, la coda comincia a svuotarsi; i token tornano ad accumularsi **dopo che il backlog è stato smaltito**, finché tutti i nuovi token servono ancora a inoltrare i dati arretrati.

10. Due code: $\rho < C < \lambda_p$

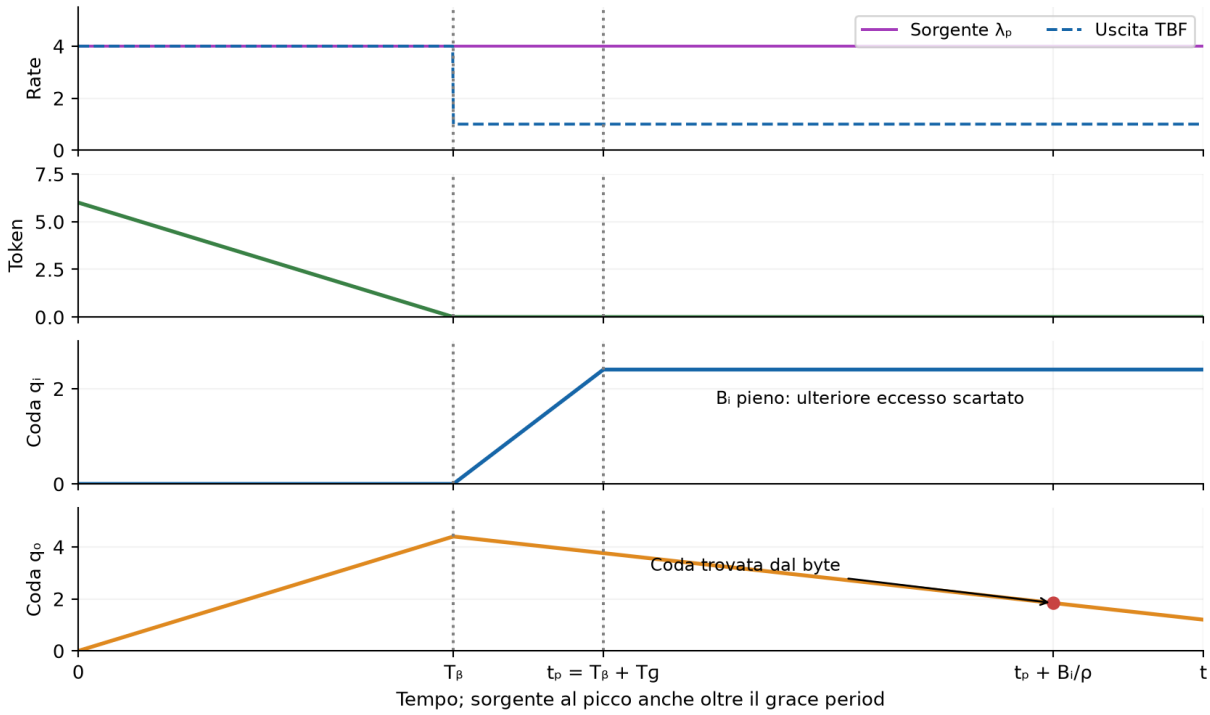


Figura 11 - Durante il grace period la coda di ingresso cresce e quella di uscita scende.

Rate e token. Sono uguali al caso precedente: il filtro passa da λ_p a ρ quando finiscono i crediti.

Coda d'ingresso. È nulla durante il burst, poi aumenta nel grace period fino a B_i.

Coda d'uscita. Durante il burst cresce a $\lambda_p - C$ e raggiunge il massimo

$$Q_o = (\lambda_p - C)T_b$$

a $t = T_b$. Usiamo Q_o per il livello massimo effettivo: la capacità fisica B_o può essere più grande. Dopo T_b il TBF fornisce $\rho < C$, quindi la coda scende al rate $C - \rho$:

$$q_o(t) = \max\{0, Q_o - (C - \rho)(t - T_b)\}, \qquad t \geq T_b,$$

finché il TBF continua a inoltrare a ρ. Nell'esempio disegnato la sorgente continua al picco, quindi questa condizione vale.

Il punto centrale della spiegazione del prof

Un dato generato al tempo t aspetta prima nella coda d'ingresso per $d_i(t)$. Arriva quindi alla coda d'uscita al tempo $t + d_i(t)$. Il ritardo complessivo è:

$$\boxed{d(t) = d_i(t) + \frac{q_o(t + d_i(t))}{C}}.$$

Non bisogna sommare le due code fotografate allo stesso istante: mentre il dato aspetta all'ingresso, la coda d'uscita si sta già svuotando. Il punto rosso nel grafico indica il livello trovato **quando il dato arriva davvero all'uscita**. Con altri parametri, a quell'istante la coda potrebbe essere già vuota.

11. Chiudere il calcolo e verificare lo SLA

Poniamo $t_p = T_b + T_g$ e scegliamo $B_i = (\lambda_p - \rho)T_g$. Il dato accettato alla fine del grace period trova la coda d'ingresso piena e aspetta:

$$d_i = \frac{B_i}{\rho}.$$

Durante $T_g + d_i$ dall'esaurimento dei token, la coda d'uscita si è ridotta. Il ritardo massimo è quindi:

$$\boxed{d_{\max} = \frac{B_i}{\rho} + \frac{\max\{0, Q_o - (C-\rho)(T_g + B_i/\rho)\}}{C}}.$$

Questo completa il procedimento spiegato dal prof negli ultimi minuti. Il capitolo dà lo stesso risultato in forma compatta:

$$\boxed{d_{\max} = \max\left\{\frac{B_i}{\rho},\ \frac{Q_o}{C} + \frac{\lambda_p - C}{C}T_g\right\}}, \qquad \rho < C < \lambda_p.$$

Perché il caso peggiore è alla fine del grace period? Durante tale periodo poniamo $\tau = t - T_b$. Finché all'arrivo resta coda in uscita, $d(t) = Q_o/C + \tau(\lambda_p/C - 1)$, crescente perché $\lambda_p > C$. Quando l'uscita è già vuota, resta $d(t) = (\lambda_p - \rho)\tau/\rho$, anch'esso crescente. Non basta confrontare le dimensioni delle due code: occorre seguire lo stesso dato attraverso entrambe.

Il contratto è soddisfatto solo se $d_{\max} \le d_{\text{SLA}}$. Nel caso di due code, basta verificare entrambe le condizioni:

$$\frac{B_i}{\rho} \le d_{\text{SLA}}, \qquad \frac{Q_o}{C} + \frac{\lambda_p - C}{C}T_g \le d_{\text{SLA}}.$$

Controlli rapidi da ricordare

- Il **policer marca**; lo **shaper ritarda** e può scartare quando il buffer è pieno.
- Il **token bucket risparmia crediti** e consente burst; il **leaky bucket conserva dati** e limita sempre l'uscita a r.
- Per una coda, **rate × tempo è un'area**: se il rate varia linearmente, compare il fattore $1/2$.
- Nei ritardi in cascata, valutare la seconda coda **all'arrivo**, non alla generazione.

Fonti e copertura dei disegni

Base: `Lesson 7.pdf`, tutte le 7 pagine; trascrizione `20 ott 2026, 08-45.transcript.txt`, da 00:00 a 01:33:54. **Integrazione mirata:** `4-QoS-TokenFilters-BookChapter.pdf`, §0.2 e parti di §0.3 relative a policer, soft shaper, burst, buffer e ritardi (§0.3.0.1-0.3.0.2). Esclusi pipeline QoS generale, CAC, scheduling e network slicing, non sviluppati in questa lezione.

I disegni sono ricreati e spiegati: p. 1 → fig. 1 e tabella dei colori; p. 2 → fig. 3-4; p. 3 → fig. 4, incluso lo zoom; p. 4 → fig. 5-6, con correzioni; p. 5 → fig. 7-8; p. 6 → fig. 8-9; p. 7 → fig. 8 e 10-11. La fig. 2 ricostruisce il grafico dei crediti descritto a voce. Gli schemi ripetuti del TBF sono riuniti nella fig. 8 e nei tre bilanci della fig. 9.

Le formule provvisorie del dente di sega sono state ricalcolate; nel ritardo a due code è corretto anche il refuso del capitolo che indica la coda d'uscita con il pedice d'ingresso. Le ultime formule del capitolo per ricavare inversamente B_i, non sviluppate dal prof e incoerenti nella versione fornita, non sono usate: la verifica qui deriva direttamente dal ritardo.